

Hackathon.IO

Documentazione Tecnica del Progetto

G. Iazzetta, C. Morelli

Marzo 2026

Indice

1	Introduzione	3
1.1	Funzionalità Principali	3
1.2	Approccio Architetture e Paradigma OOP	3
2	Il Modello (Model)	4
2.1	Gerarchia degli Utenti ed Ereditarietà	4
2.2	Gestione degli Eventi e Design Pattern	4
2.3	Le Entità di Supporto e Valutazione	4
3	Gestione dei Dati (DAO)	5
3.1	Sicurezza e Gestione delle Risorse	5
3.2	Risoluzione Dinamica degli Utenti	5
3.3	Aggregazione dei Dati ed Elaborazioni Complesse	5
4	La Logica di Controllo (Controller)	6
4.1	Disaccoppiamento tramite Interfacce	6
4.2	Gestione dello Stato e Polimorfismo	6
4.3	Validazione e Regole di Business	6
4.4	Gestione degli Errori e delle Eccezioni	7
5	L'Interfaccia Grafica (GUI)	7
5.1	Navigazione a Singola Finestra (CardLayout)	7
5.2	Un'Interfaccia Dinamica e Intelligente	7
5.3	User Experience e Rendering HTML	7
5.4	Nota Architetture: Il rapporto pragmatico tra GUI e Model	8
6	Classi di Supporto (Utility)	8
6.1	Gestione Centralizzata del Tema: <code>UIColors</code>	9
6.2	Ereditarietà e Grafica Personalizzata: <code>RoundedPanel</code>	9
7	Gestione degli Errori	9
7.1	Ereditarietà e Checked Exceptions	10
7.2	Separazione delle Responsabilità (Throw e Catch)	10
8	Conclusioni e Sviluppi Futuri	10
8.1	Considerazioni Architetture	10
8.2	Possibili Sviluppi Futuri	10

1 Introduzione

Il presente documento illustra l'architettura, le scelte implementative e i *design pattern* relativi al progetto "Hackathon.IO", un applicativo desktop sviluppato in linguaggio Java per l'esame di Programmazione Orientata agli Oggetti.

Lo scopo del software è fornire una piattaforma gestionale (in versione demo) per l'organizzazione e la partecipazione a tornei di tipo *Hackathon*. Il sistema è stato modellato per riflettere le dinamiche reali di questo genere di competizioni, permettendo agli utenti di registrarsi e di assumere ruoli specifici e dinamici all'interno dell'ecosistema: **Organizzatori** (creatori e gestori dell'evento), **Partecipanti** (membri di una squadra in competizione) e **Giudici** (esperti incaricati della valutazione).

1.1 Funzionalità Principali

L'applicativo copre l'intero ciclo di vita di un hackathon, offrendo le seguenti funzionalità cardine:

- **Gestione Utente:** Registrazione e autenticazione sicura, con risoluzione dinamica dei ruoli per sbloccare funzionalità specifiche nell'interfaccia.
- **Organizzazione Eventi:** Creazione di nuovi hackathon definendone parametri cruciali come finestre temporali, limiti di capienza e descrizioni delle tracce da sviluppare.
- **Gestione Squadre (Team):** Creazione di gruppi di lavoro protetti da codici di accesso univoci, per semplificare l'aggregazione dei partecipanti.
- **Sottomissione Progetti:** Un'area dedicata all'upload di risorse (es. link a repository GitHub, demo o presentazioni) da parte dei team in gara.
- **Sistema di Valutazione:** Un modulo riservato alla giuria per la revisione dei progetti, il rilascio di feedback testuali dettagliati e l'assegnazione di un punteggio quantitativo (da 0 a 10).
- **Classifiche (Leaderboard):** Generazione automatica della classifica finale basata sulla media dei voti ricevuti, accessibile solo al termine dell'evento.

1.2 Approccio Architeturale e Paradigma OOP

Dal punto di vista ingegneristico, lo sviluppo del sistema è stato guidato dall'applicazione dei pilastri della programmazione orientata agli oggetti: **incapsulamento**, **ereditarietà** e **polimorfismo**.

Per garantire la stesura di un codice manutenibile, l'architettura complessiva si fonda sul pattern **BCE (Boundary-Control-Entity)**. Questa netta suddivisione logica dei layer ha permesso di disaccoppiare totalmente l'interfaccia grafica (Boundary) dai dettagli tecnici legati alla persistenza sul database (Entity e DAO), centralizzando la logica di business e le regole di validazione in un unico *Orchestratore* (Control).

Nelle sezioni successive verranno analizzati nel dettaglio i singoli livelli di questa architettura, mettendo in luce le problematiche affrontate, le soluzioni adottate e il corretto utilizzo dei *design pattern* creazionali e strutturali.

2 Il Modello (Model)

Il livello Model dell'applicazione contiene le classi che rappresentano i dati e le entità fondamentali del nostro dominio (come gli utenti, eventi e i team). Seguendo i principi base della programmazione orientata agli oggetti (OOP), tutte le classi sono state progettate garantendo un corretto **incapsulamento**: gli attributi sono definiti come privati per proteggere lo stato interno dell'oggetto, ed è possibile leggerli o modificarli esclusivamente tramite appositi metodi pubblici *getter* e *setter*.

2.1 Gerarchia degli Utenti ed Ereditarietà

Per modellare i diversi tipi di attori che interagiscono con il sistema, abbiamo sfruttato ampiamente il concetto di **ereditarietà**, evitando così di duplicare inutilmente il codice:

- **User**: È la *superclasse* di base. Raccoglie le informazioni condivise da chiunque si registri sulla piattaforma, ovvero l'identificativo univoco, il nome, l'email e la password.
- **Organizer e Judge**: Sono *sottoclassi* che estendono **User**. Ereditano tutti i dati anagrafici, ma aggiungono un riferimento specifico (tramite l'ID) all'hackathon che stanno organizzando o giudicando.
- **Participant**: Anch'essa estende **User**, ma specializza l'utente aggiungendo le informazioni necessarie per la gara, come l'appartenenza a una squadra (`teamId`).

2.2 Gestione degli Eventi e Design Pattern

L'entità centrale del progetto è rappresentata dalla classe **Hackathon**. Poiché un evento è descritto da numerosi attributi (date di inizio e fine, limiti di partecipanti, descrizione del problema, ecc.), utilizzare un costruttore tradizionale avrebbe richiesto una lista di parametri lunghissima e difficile da gestire.

Per risolvere questo problema in modo elegante, abbiamo implementato il **Builder Pattern**. Attraverso una classe interna di supporto (**Hackathon.Builder**), è possibile istanziare l'oggetto passo dopo passo in modo molto più intuitivo, riducendo drasticamente il rischio di inserire i parametri nell'ordine sbagliato.

2.3 Le Entità di Supporto e Valutazione

Le altre classi del modello servono a gestire lo svolgimento pratico del torneo e le relazioni tra gli utenti:

- **Team, Document e Vote**: Queste classi rappresentano rispettivamente le squadre create dai partecipanti, le risorse (file o link) consegnate e i voti numerici assegnati dai giudici. Contengono gli identificativi necessari per collegarsi logicamente agli utenti e agli eventi corrispondenti.
- **Feedback**: Rappresenta il commento testuale lasciato da un giudice su un documento. In questa classe abbiamo adottato un accorgimento pratico: oltre a salvare l'ID del giudice, abbiamo inserito un campo apposito per il suo nome (`judgeName`). Questo ci permette di recuperare e mostrare facilmente a schermo chi ha scritto la recensione, rendendo l'applicazione più efficiente durante l'interazione con il database.

3 Gestione dei Dati (DAO)

Il livello di accesso ai dati è stato progettato seguendo il pattern **DAO (Data Access Object)**. L'obiettivo principale di questo livello è separare in modo netto la logica di business dell'applicazione (gestita dal Controller) dai dettagli tecnici relativi al salvataggio e al recupero delle informazioni sul database.

Per garantire la massima flessibilità, ogni entità del sistema (Documenti, Feedback, Utenti, ecc.) è gestita da due componenti distinti:

1. Un'**Interfaccia** (es. `UserDAO`), che definisce un "contratto", ovvero l'elenco delle operazioni possibili (come salvare, cercare o eliminare un dato) senza specificare come queste debbano essere eseguite.
2. Una **Classe di Implementazione** (es. `UserDAOImpl`), che contiene il codice effettivo per comunicare con il database PostgreSQL.

Questo approccio permette al resto dell'applicazione di dialogare esclusivamente con le interfacce, ignorando la complessità del database sottostante.

3.1 Sicurezza e Gestione delle Risorse

Tutte le operazioni di interazione con il database sono state scritte ponendo particolare attenzione alla sicurezza e all'efficienza. Al fine di prevenire attacchi di tipo *SQL Injection*, ogni parametro fornito dall'utente viene inserito nelle query utilizzando i **PreparedStatement**. Inoltre, per evitare "perdite di memoria" (Resource Leaks), l'apertura e la chiusura delle connessioni al database sono gestite tramite il costrutto Java **try-with-resources**, che garantisce il rilascio automatico delle risorse (connessioni e risultati) non appena l'operazione è conclusa o in caso di errore.

3.2 Risoluzione Dinamica degli Utenti

Uno degli aspetti più interessanti del DAO riguarda la gestione del login all'interno di `UserDAOImpl`. Quando un utente inserisce le proprie credenziali, il sistema non si limita a verificare la correttezza della password. Sfruttando le potenzialità del **Polimorfismo**, il DAO interroga il database per scoprire il ruolo dell'utente (se è iscritto a un team, se è un giudice o un organizzatore) e restituisce all'applicazione l'oggetto specifico corretto (`Participant`, `Judge` o `Organizer`). In questo modo, l'applicazione sa fin da subito quali permessi accordare.

3.3 Aggregazione dei Dati ed Elaborazioni Complesse

Il livello DAO si fa carico anche di alleggerire il carico di lavoro del Controller, eseguendo direttamente le elaborazioni sui dati sfruttando le potenzialità del linguaggio SQL. Alcuni esempi di questa ottimizzazione includono:

- **Recupero mirato di informazioni (JOIN):** Quando si richiede l'elenco dei feedback per un documento (`FeedbackDAOImpl`), il sistema recupera in un solo passaggio sia il testo del commento sia il nome del giudice autore, evitando di dover fare una seconda richiesta al database.
- **Statistiche e Classifiche:** Nel `VoteDAOImpl`, il calcolo della classifica finale (Leaderboard) non viene fatto scaricando tutti i voti e calcolandone la media in Java. Al contrario, viene utilizzata una singola query SQL avanzata che aggrega i voti, calcola le medie parziali e restituisce un elenco testuale già ordinato dalla prima all'ultima posizione, pronto per essere mostrato a schermo.

4 La Logica di Controllo (Controller)

Il livello *Control* rappresenta il vero e proprio "cervello" dell'applicazione. Seguendo esplicitamente le direttive di progetto, l'intera logica di business e l'orchestrazione del sistema sono state centralizzate all'interno di un'unica classe: il **Controller**.

Questa scelta progettuale trasforma il Controller in un punto di accesso unico e centralizzato, che fa da mediatore tra ciò che l'utente vede a schermo (la GUI) e i dati salvati nel database (i DAO). La GUI non comunica mai direttamente con il database, e viceversa.

4.1 Disaccoppiamento tramite Interfacce

Per garantire un codice flessibile e manutenibile, il Controller è stato progettato seguendo il **Principio di Inversione delle Dipendenze**. Osservando la struttura della classe, noteremo che il Controller possiede dei riferimenti ai vari DAO (come `UserDAO` o `TeamDAO`), ma questi sono definiti esclusivamente tramite le loro *interfacce*, e non tramite le classi concrete (come `UserDAOImpl`).

Questo significa che il Controller "sa" quali azioni può far compiere al database, ma "ignora" completamente come queste vengano implementate tecnicamente (in questo caso, tramite PostgreSQL). Se un domani volessimo cambiare il database salvando i dati su file di testo, non dovremmo modificare nemmeno una riga del Controller.

4.2 Gestione dello Stato e Polimorfismo

Il Controller ha il compito fondamentale di mantenere lo stato dell'applicazione durante la sessione utente. Lo fa principalmente attraverso la variabile `loggedInUser`, che memorizza chi sta attualmente utilizzando il sistema.

Il fiore all'occhiello di questa gestione è il metodo di login (`resolveActualUserRole`). Quando un utente effettua l'accesso, il Controller non si limita a caricare un utente generico. Attraverso una serie di controlli, verifica il ruolo specifico della persona (se ha un team, se è un giudice o un organizzatore) e "trasforma" l'oggetto base in un'istanza della sottoclasse corretta (`Participant`, `Judge` o `Organizer`). Grazie al **Polimorfismo**, il resto del codice non avrà bisogno di fare continui e scomodi controlli sul tipo di utente (*casting* espliciti o cascate di `if-else`): le autorizzazioni vengono gestite in modo naturale e intrinseco dall'oggetto stesso.

4.3 Validazione e Regole di Business

Il Controller è il garante delle "regole del gioco". Prima di eseguire qualsiasi operazione delegandola al DAO, verifica che le condizioni logiche siano rispettate. Alcuni esempi di regole gestite a questo livello sono:

- **Creazione Eventi:** Un utente può creare un nuovo hackathon (`canUserCreateHackathon`) solo se non sta già partecipando ad un altro evento con un ruolo attivo.
- **Gestione Squadre:** Per iscriversi a un team è necessario fornire il codice d'accesso corretto. Inoltre, solo chi ha il ruolo di Partecipante può caricare i documenti del progetto.
- **Classifiche:** Le classifiche finali (`getFinalRanking`) vengono calcolate e rese visibili esclusivamente se la data di fine dell'evento è già stata superata, prevenendo visualizzazioni premature da parte dei partecipanti.

4.4 Gestione degli Errori e delle Eccezioni

Coerentemente con il pattern BCE, il Controller **non** si occupa di mostrare messaggi di errore a schermo (es. tramite stampe su console o finestre di dialogo), poiché non è il suo compito. Quando intercetta una situazione anomala (es. campi vuoti o credenziali errate), il Controller solleva delle **Eccezioni** personalizzate (come `UserNotFoundException` o `BlankFieldException`) e le "lancia" verso l'alto. Sarà poi la GUI a intercettare queste eccezioni e a decidere come mostrarle all'utente in modo elegante.

5 L'Interfaccia Grafica (GUI)

Il livello *Boundary* costituisce l'interfaccia visiva dell'applicazione, ovvero l'unico punto di contatto diretto con l'utente. L'intera GUI è stata sviluppata utilizzando il framework standard **Java Swing**, coadiuvato dal GUI Designer integrato nell'IDE (IntelliJ IDEA) per la disposizione spaziale dei componenti.

In rigoroso ossequio al pattern architetturale scelto, le classi della GUI (come le varie finestre e i pannelli) ignorano totalmente l'esistenza del database e del livello DAO. Il loro unico compito è "ascoltare" le azioni dell'utente (tramite gli *Event Listener*) e delegare l'esecuzione della logica al Controller, mostrandone poi i risultati a schermo.

5.1 Navigazione a Singola Finestra (CardLayout)

Per garantire un'esperienza d'uso fluida e moderna, abbiamo evitato l'approccio tradizionale che prevede la continua apertura e chiusura di decine di finestre diverse. Abbiamo invece sfruttato le potenzialità del `CardLayout`.

Sia la fase di autenticazione (`AuthFrame`) che l'applicativo principale (`MainFrame`) agiscono come "contenitori fissi". Al loro interno, i vari pannelli funzionali (`DashboardCardPanel`, `TeamCardPanel`, ecc.) vengono sovrapposti e "sfogliati" come le carte di un mazzo in base alla sezione selezionata. Questo approccio riduce il carico sulla memoria e restituisce un *look and feel* molto simile a quello delle moderne *Single Page Application* (SPA) del mondo web.

5.2 Un'Interfaccia Dinamica e Intelligente

Sfruttando il lavoro di identificazione svolto dal Controller (Polimorfismo), l'interfaccia grafica è in grado di **adattarsi dinamicamente** in base al ruolo di chi ha effettuato l'accesso. Piuttosto che riempire lo schermo di pulsanti disabilitati o bloccati da messaggi d'errore, la GUI nasconde attivamente le sezioni non pertinenti:

- **Menu Contestuale:** Nella barra laterale del `MainFrame`, la voce "Manage" compare esclusivamente se l'utente è un Organizzatore, mentre la voce "Evaluation" è visibile solo ai Giudici.
- **Azioni Limitate:** Nella `DashboardCardPanel`, il pulsante per creare un nuovo hackathon appare solamente se l'utente loggato è un utente base, scomparendo se quest'ultimo possiede già un ruolo attivo in un evento. Analogamente, la possibilità di caricare documenti (`DocumentUploadDialog`) viene sbloccata solo per i Partecipanti.

5.3 User Experience e Rendering HTML

Particolare attenzione è stata posta alla *User Experience* (UX) e all'estetica. Per evitare la rigidità visiva tipica di Swing, sono stati implementati componenti personalizzati (come i `RoundedPanel`) che reagiscono al passaggio del mouse con effetti di *hovering* (cambi di colore dinamici).

Una sfida interessante è stata la visualizzazione dello storico dei feedback lasciati dai giudici sui documenti caricati. Invece di limitarci a un'anonima area di testo semplice (`JTextArea`), abbiamo utilizzato un componente avanzato (`JEditorPane`) configurato per interpretare codice **HTML**. Prima di essere mostrati, i dati estratti dal *Controller* vengono "impacchettati" dinamicamente in una struttura HTML all'interno della GUI. Questo ha permesso di utilizzare font differenziati, colori gerarchici (titoli in rosso, nomi dei giudici in blu notte) e linee di separazione, rendendo la lettura dei commenti estremamente chiara e professionale.

5.4 Nota Architetture: Il rapporto pragmatico tra GUI e Model

Esaminando i file sorgente del livello *Boundary*, si osserva la frequente importazione delle classi appartenenti al pacchetto `model` (quali `Hackathon`, `User` o `Team`). Sebbene le interpretazioni più dogmatiche del pattern **BCE (Boundary-Control-Entity)** suggeriscano di isolare completamente la View dal Model tramite l'uso di *Data Transfer Object* (DTO) intermedi, per questo progetto si è optato per un approccio ingegneristico pragmatico, mirato a prevenire l'over-engineering e il fenomeno della *Class Explosion*.

In questa architettura, le entità del dominio vengono fornite alla GUI dal *Controller* e utilizzate come **“Contratti di Dati” in sola lettura**. Questa scelta è supportata da tre precisi vantaggi strutturali, riscontrabili direttamente nell'implementazione:

- **Prevenzione dei *Data Clumps* (`HackathonCardPanel`):** Senza l'importazione del Model, per popolare la vista di dettaglio di un evento, il *Controller* avrebbe dovuto restituire decine di variabili primitive slegate tra loro. Al contrario, il metodo `populateFields(Hackathon h)` riceve l'entità completa, garantendo che i dati viaggino in uno stato aggregato e coerente.
- **Centralizzazione della Business Logic (`JudgeManageCardPanel` e `TeamCardPanel`):** La GUI deve adattare i propri componenti (es. disabilitare l'upload o la valutazione) in base allo stato temporale dell'evento. Invece di duplicare la logica di confronto delle date nella *Boundary*, l'interfaccia interroga direttamente il Model invocando i metodi `h.isStarted()` e `h.isEnded()`. La logica di dominio resta così incapsulata nell'entità.
- **Routing Polimorfico e RBAC (`MainFrame`):** L'importazione delle sottoclassi di `User` (`Organizer`, `Judge`, `Participant`) ha permesso di implementare un sistema di *Role-Based Access Control* (RBAC) basato sul **Polimorfismo**. Il menu di navigazione si adatta dinamicamente all'utente loggato valutando l'istanza a runtime (es. `user instanceof Organizer`), evitando l'uso di fragili flag testuali o booleani.

A garanzia del mantenimento del **disaccoppiamento logico**, è sufficiente osservare il flusso inverso dei dati: quando la GUI invia informazioni al sistema (ad esempio, durante la creazione di un evento nel `DashboardCardPanel`), non istanzia mai oggetti del Model in autonomia. Essa estrae i valori primitivi dai campi di testo e li passa come argomenti ai metodi del *Controller*. Sarà compito esclusivo di quest'ultimo validare gli input, costruire le entità tramite il *Builder Pattern* e orchestrare la persistenza tramite i *DAO*.

6 Classi di Supporto (Utility)

Per mantenere il codice della GUI pulito e modulare, evitando di appesantire le singole schermate con dettagli puramente estetici o configurazioni ripetitive, è stato introdotto un pacchetto dedicato alle *Utility*. Queste classi agiscono come strumenti di supporto trasversali per l'intera applicazione.

6.1 Gestione Centralizzata del Tema: UIColors

La classe `UIColors` è stata creata per definire la *palette* cromatica ufficiale dell'applicativo (come il rosso carminio e il blu notte).

Invece di spargere per tutto il codice i codici RGB dei colori (pratica nota come l'uso di *Magic Numbers*, considerata un *anti-pattern*), i colori sono stati centralizzati qui tramite costanti (`public static final`). Questa scelta rispetta a pieno il principio **DRY (Don't Repeat Yourself)**: se in futuro si decidesse di modificare la tonalità di blu dell'applicazione, sarà sufficiente cambiare un singolo valore all'interno di questa classe, e l'intera interfaccia grafica (pulsanti, sfondi e testi) si aggiornerà automaticamente di conseguenza.

6.2 Ereditarietà e Grafica Personalizzata: RoundedPanel

L'interfaccia standard di Java Swing offre componenti dalle forme tipicamente squadrate e rigide. Per garantire un design più moderno e accattivante, abbiamo realizzato la classe `RoundedPanel`.

Dal punto di vista della Programmazione Orientata agli Oggetti, questa classe rappresenta un ottimo esempio di **Ereditarietà** e **Overriding** (sovrascrittura dei metodi):

- **Ereditarietà:** La classe estende (`extends`) un normale `JPanel` di Swing. In questo modo eredita gratuitamente tutte le funzionalità base di un pannello (la capacità di contenere altri elementi, di ascoltare i click del mouse, ecc.).
- **Overriding:** Per modificare l'aspetto visivo, abbiamo "sovrascritto" il metodo `paintComponent(Graphics g)`. Sfruttando le librerie `Graphics2D`, il pannello disegna autonomamente i propri bordi arrotondati, applicando persino dei filtri di *Anti-Aliasing* per smussare le scalettature dei pixel sui bordi curvi.

In questo modo, chi sviluppa le singole schermate (come la Dashboard o il Login) può utilizzare il `RoundedPanel` esattamente come un pannello normale, ignorando del tutto la complessa logica matematica e grafica che lavora "dietro le quinte" per smussarne gli angoli.

7 Gestione degli Errori

Un'applicazione robusta non deve limitarsi a funzionare nei casi ideali, ma deve saper gestire le anomalie in modo elegante, evitando crash improvvisi e fornendo feedback chiari all'utente. Per raggiungere questo obiettivo, abbiamo implementato un sistema di **gestione degli errori tramite Eccezioni Personalizzate** (Custom Exceptions).

Invece di affidarci esclusivamente alle eccezioni standard di Java (come `IllegalArgumentException` o `RuntimeException`), che risultano spesso troppo generiche o puramente tecniche, abbiamo creato un pacchetto dedicato (`exceptions`) contenente classi specifiche per il dominio del nostro problema. Alcuni esempi includono:

- Errori di validazione dell'input: `BlankFieldException`, `PasswordsDoNotMatchException`.
- Violazioni delle regole di business: `AlreadyPartOfATeamException`, `MaxLimitReachedException`, `InvalidTimeWindowException`.
- Conflitti nei dati utente: `EmailAlreadyTakenException`, `UsernameAlreadyTakenException`, `UserNotFoundException`.
- Errori relativi ai permessi e ruoli: `UserIsAJudgeException`, `UserIsAnOrganizerException`.

7.1 Ereditarietà e Checked Exceptions

Dal punto di vista della Programmazione Orientata agli Oggetti, tutte queste classi sfruttano l'**ereditarietà** estendendo direttamente la classe base `Exception` di Java. Questa scelta progettuale le classifica come *Checked Exceptions*. Ciò significa che il compilatore Java ci obbliga rigorosamente a gestirle: ogni volta che un metodo rischia di generare uno di questi errori, deve dichiararlo esplicitamente (tramite la parola chiave `throws`), costringendo il programmatore a racchiudere la chiamata in un blocco `try-catch`.

Inoltre, per garantire flessibilità, i costruttori di queste eccezioni richiamano il costruttore della superclasse tramite `super()`. Alcune eccezioni passano un messaggio di errore fisso e predefinito (es. `super("Passwords do not match.")`), mentre altre permettono di iniettare dinamicamente un messaggio personalizzato a seconda del contesto.

7.2 Separazione delle Responsabilità (Throw e Catch)

L'aspetto più importante di questa implementazione è il modo in cui le eccezioni interagiscono con l'architettura BCE. Il **Controller** (livello logico) ha il compito di validare i dati e applicare le regole: se qualcosa va storto, solleva (lancia) l'eccezione interrompendo il flusso. Tuttavia, il Controller non stampa mai l'errore a schermo. È la **GUI** (livello Boundary) che, invocando il Controller all'interno di un blocco `try-catch`, intercetta (cattura) l'eccezione e decide come mostrarla all'utente, traducendo l'oggetto tecnico in finestre di dialogo grafiche (i classici *pop-up* rossi di errore) facilmente comprensibili.

8 Conclusioni e Sviluppi Futuri

La realizzazione del progetto "Hackathon.IO" ha rappresentato un'ottima opportunità per tradurre i concetti teorici della Programmazione Orientata agli Oggetti in un'applicazione concreta e funzionante.

8.1 Considerazioni Architettureali

Il più grande traguardo di questo progetto non risiede solo nelle funzionalità implementate, ma nel *come* sono state strutturate. L'adozione rigorosa del pattern BCE (Boundary-Control-Entity) e del pattern DAO ha dimostrato sul campo i vantaggi del disaccoppiamento del codice.

Separare la logica di accesso al database (PostgreSQL) dalla logica di business (Controller) e dall'interfaccia grafica (Swing) ci ha permesso di lavorare in modo molto più ordinato. Modificare l'estetica di un pannello o aggiornare una query SQL non ha mai richiesto la riscrittura a cascata delle altre classi, confermando l'efficacia del principio di Singola Responsabilità (Single Responsibility Principle).

8.2 Possibili Sviluppi Futuri

Sebbene l'applicativo si presenti come una demo, l'architettura scalabile adottata si presta a diverse espansioni future. Alcuni dei possibili miglioramenti includono:

- **Esportazione dei Dati:** Implementare funzionalità per permettere all'organizzatore di scaricare la classifica finale o l'elenco dei partecipanti in formati standard come PDF o CSV.
- **Integrazione Notifiche:** Aggiungere un sistema (simulato o reale) per l'invio di notifiche via e-mail ai partecipanti, ad esempio per confermare l'iscrizione a un team o avvisare della pubblicazione di un nuovo feedback da parte dei giudici.

In sintesi, il progetto non è stato solo un esercizio di scrittura di codice Java, ma una vera e propria palestra di *Software Engineering*, che ci ha insegnato l'importanza vitale di una buona fase di progettazione prima di scrivere la primissima riga di codice.